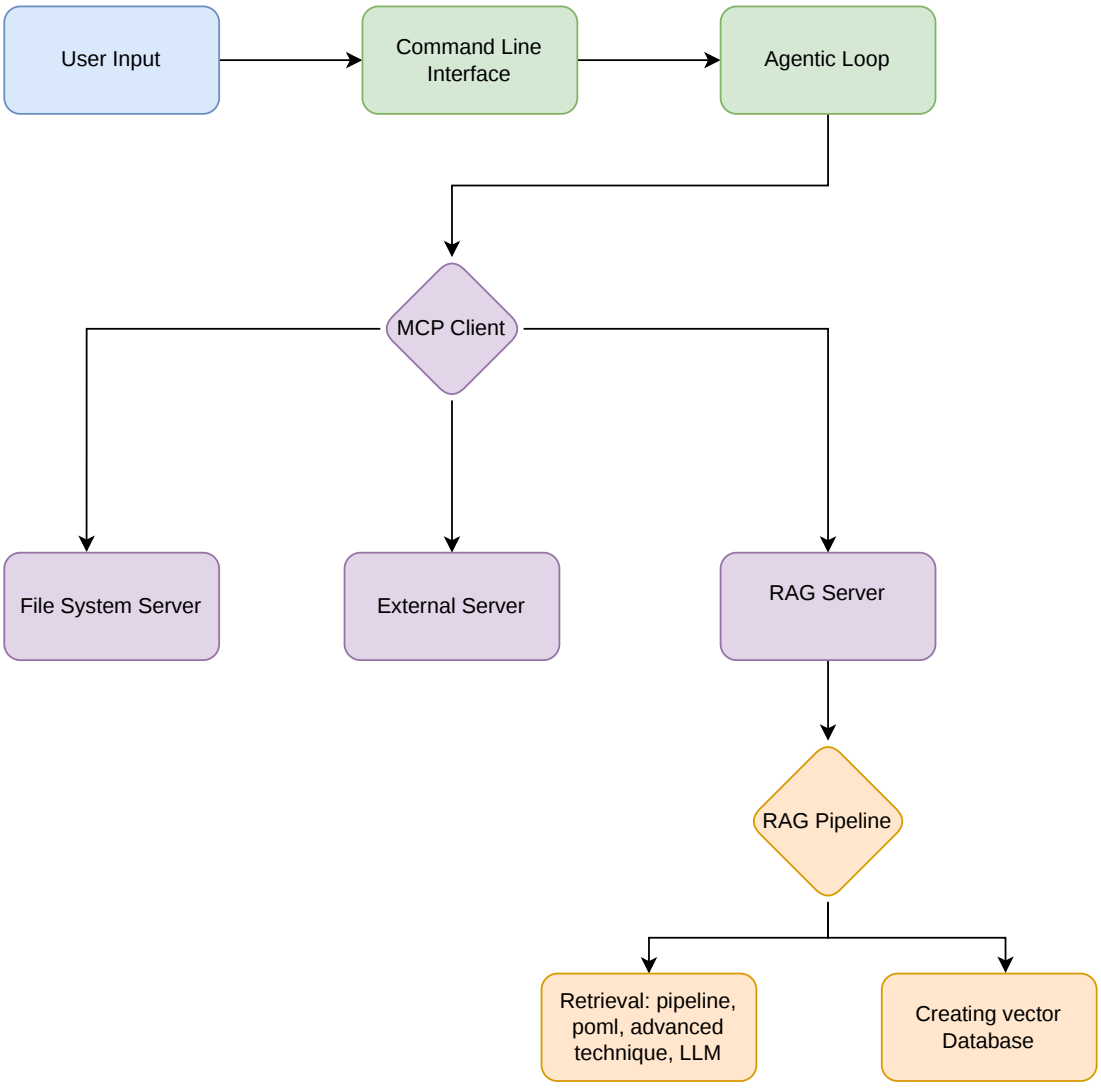
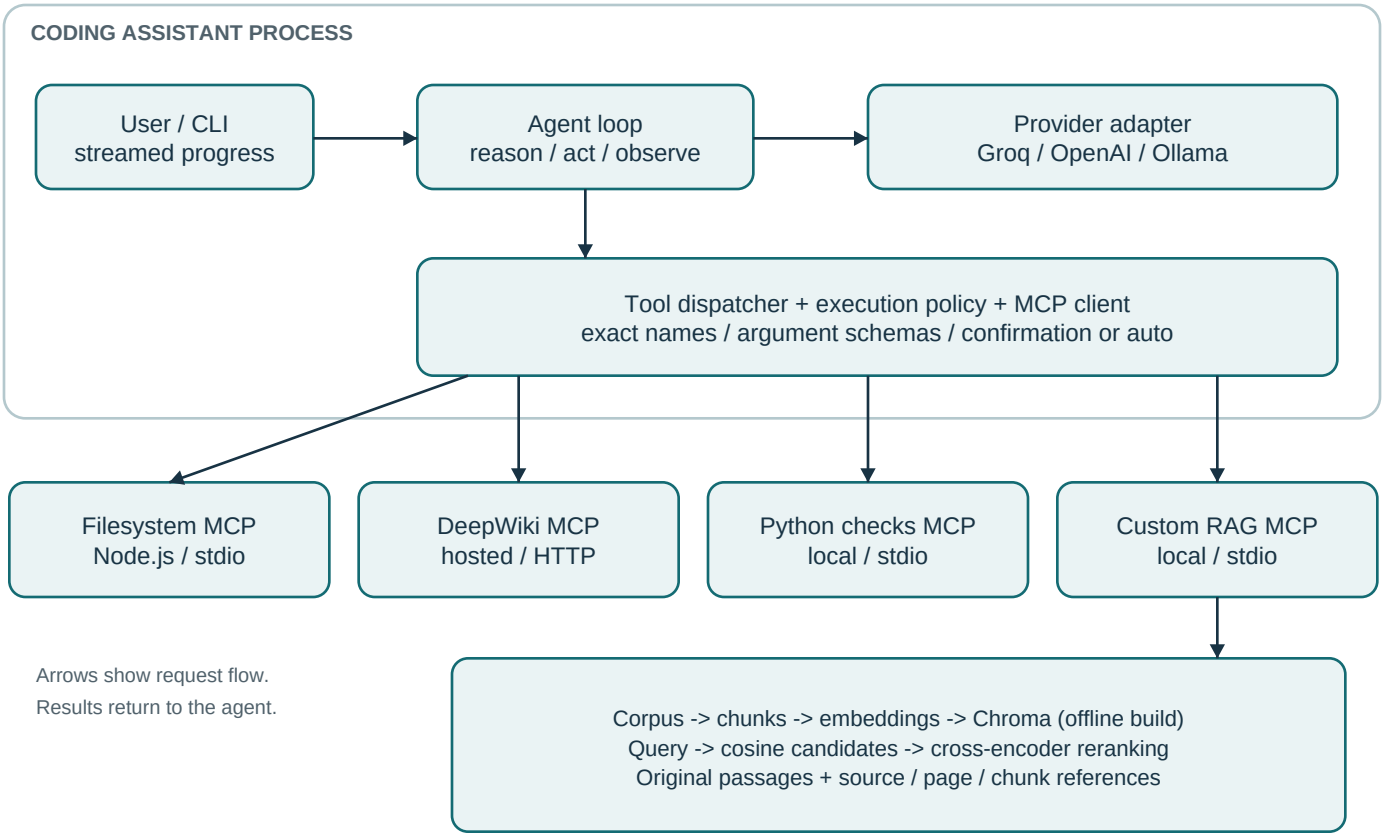
Initial Architecture



5. Updated architecture

The original planning diagram appears on page 5. This diagram shows the final system, including reliability changes and the Python-check server. It preserves the planned CLI, agent, MCP client, and three required server roles.



What changed from the plan, and why

Model boundary made explicit. Groq, OpenAI, and Ollama adapters sit outside the agent's coordination logic so models can change without replacing the tool pipeline. The planning diagram did not show this boundary.

Execution checks added explicitly. Discovered tool names, schemas, and confirmation policy sit before MCP calls. Provider-side aliases, context budgeting, and retries address the failures observed during evaluation.

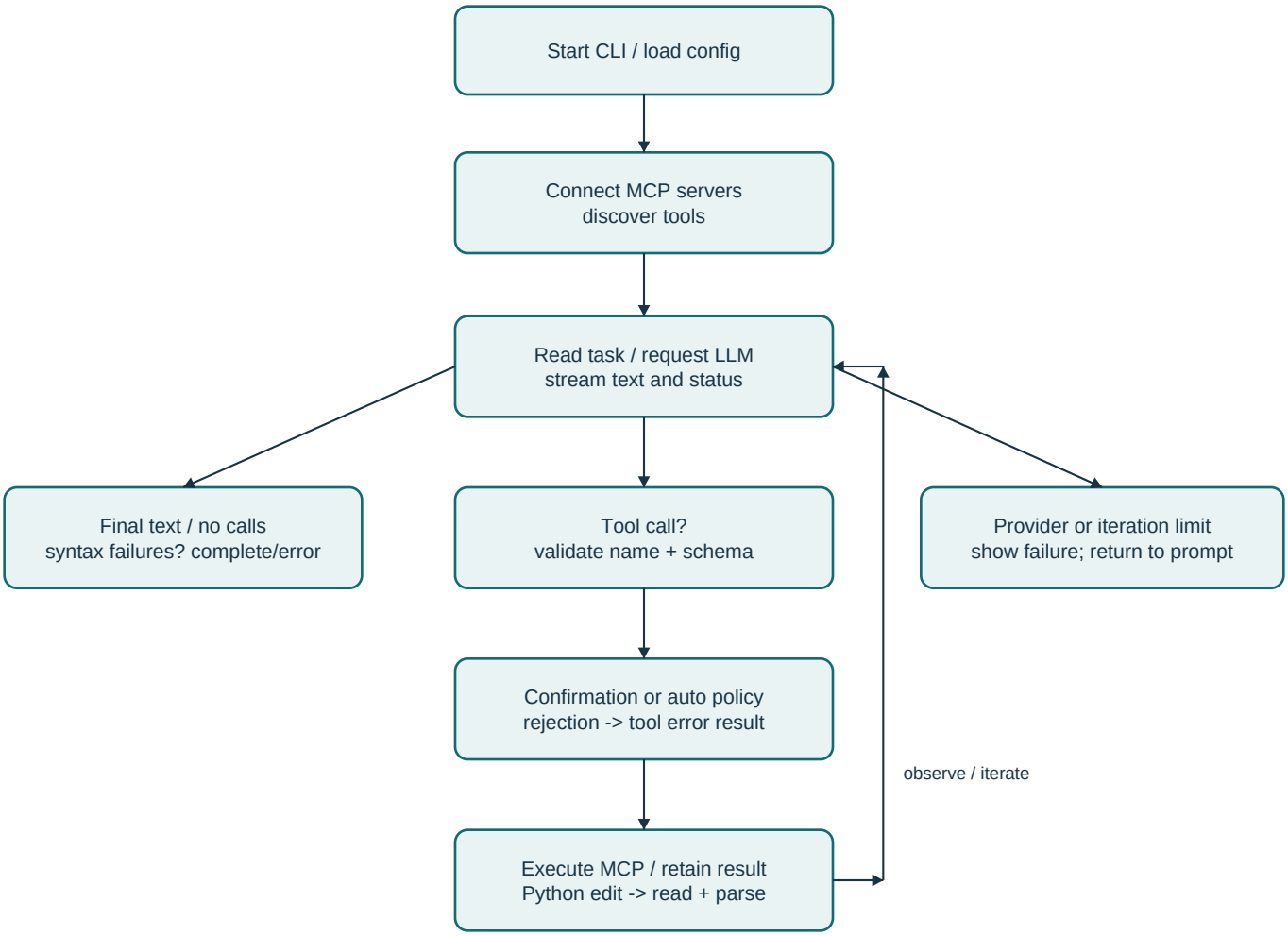
RAG design made concrete. The plan's generic retrieval/vector-database branches became an offline ingestion/index build and a query-time cosine-plus-cross-encoder pipeline. Answer generation remains in the main agent/provider; the RAG server returns evidence. Chroma and reranking were chosen to reuse vectors and improve passage ordering.

Deployment boundaries clarified. Filesystem and RAG are local stdio processes; DeepWiki is hosted over HTTP. The RAG executable follows the running Python interpreter to avoid missing dependencies. These refinements explain how the original architecture became executable.

Command execution added. A separate local Python-check MCP server runs trusted scripts/tests with per-call policy, target validation, time limits, credential-environment omission, and bounded returned output. This fills the assignment's command-execution capability without adding a shell interface. It is not an OS sandbox.

6. Task state diagram

The state flow combines user input, model decisions, validation, approval, MCP execution, feedback, and completion. See docs/diagrams.md for editable Mermaid sources and error branches.



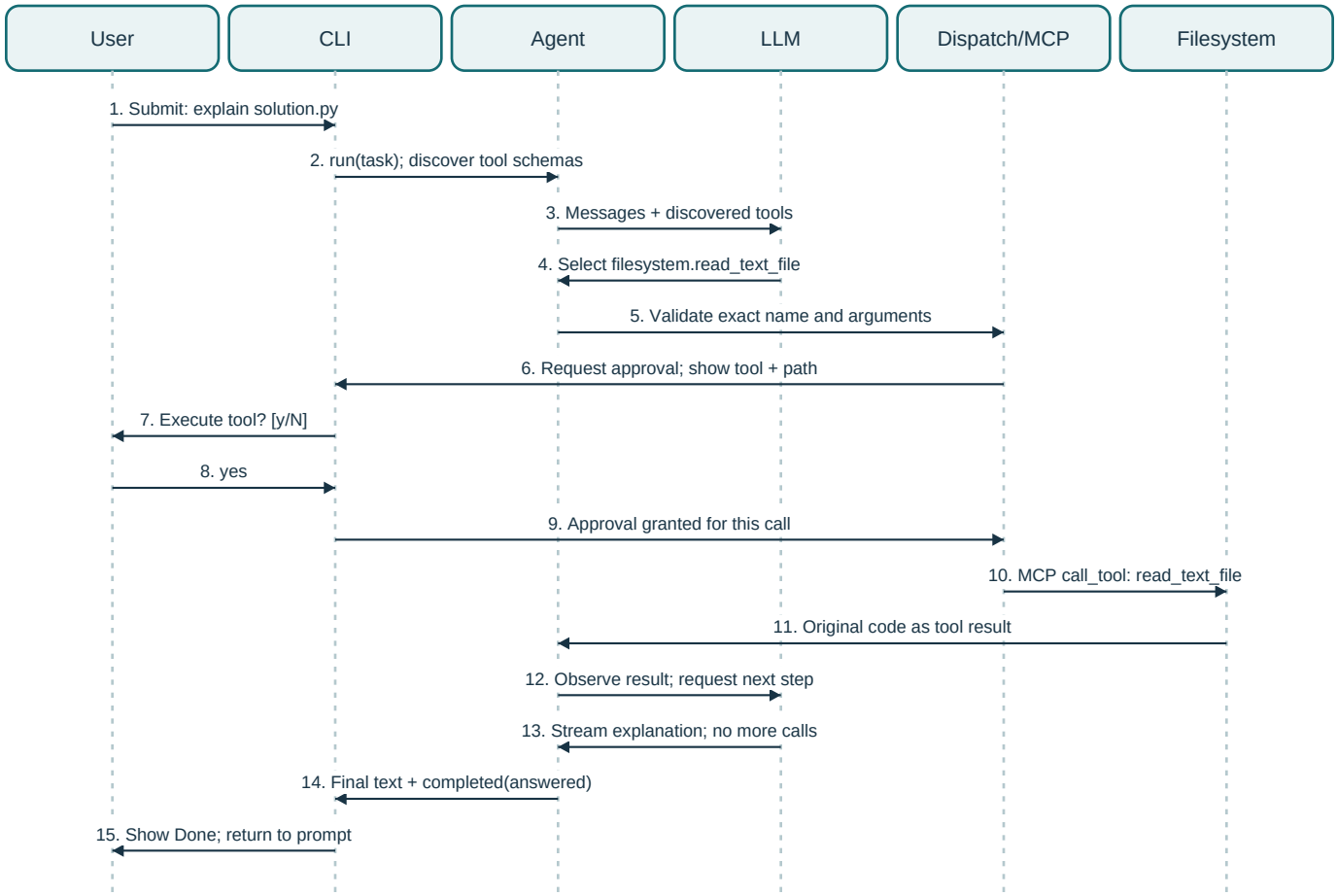
Completed or failed task -> next prompt. /exit or interruption -> close clients and exit.

Invalid calls return errors without execution. Groq retries are bounded inside its adapter.

The CLI can run many tasks in one session. One-shot mode exits after its task. All clients close on exit. The LLM sees full tool results through the agent; the Groq adapter may compact the request view. Python syntax feedback is added after writes, while optional behavioral execution is a distinct checks.run_python_check tool call.

Inspect existing code

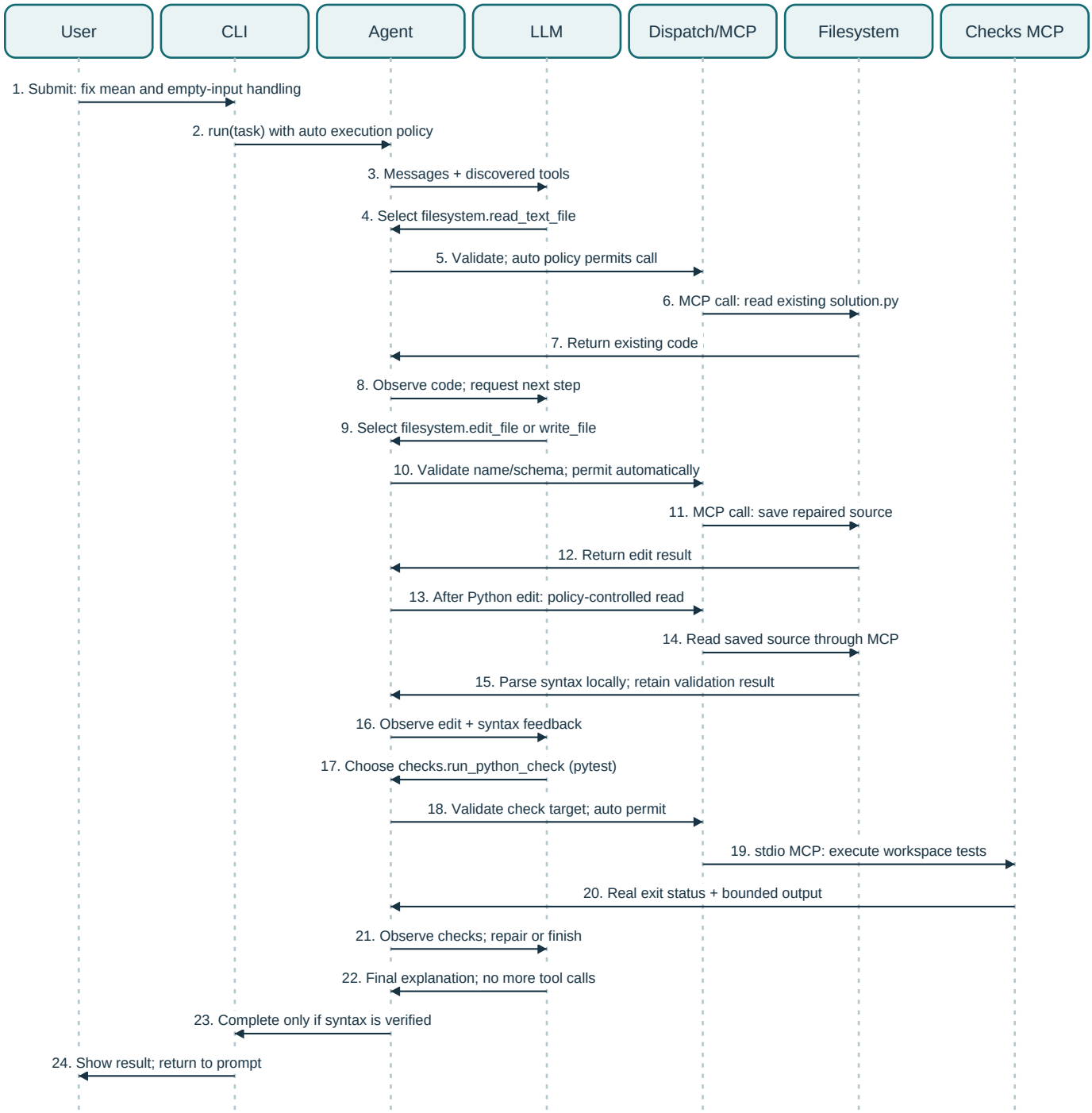
Confirmation mode | operation 1



If approval is rejected, no filesystem call occurs. The rejection is returned to the LLM as a tool error, so it can explain or revise its plan.

Repair a Python bug

Automatic mode | operation 2

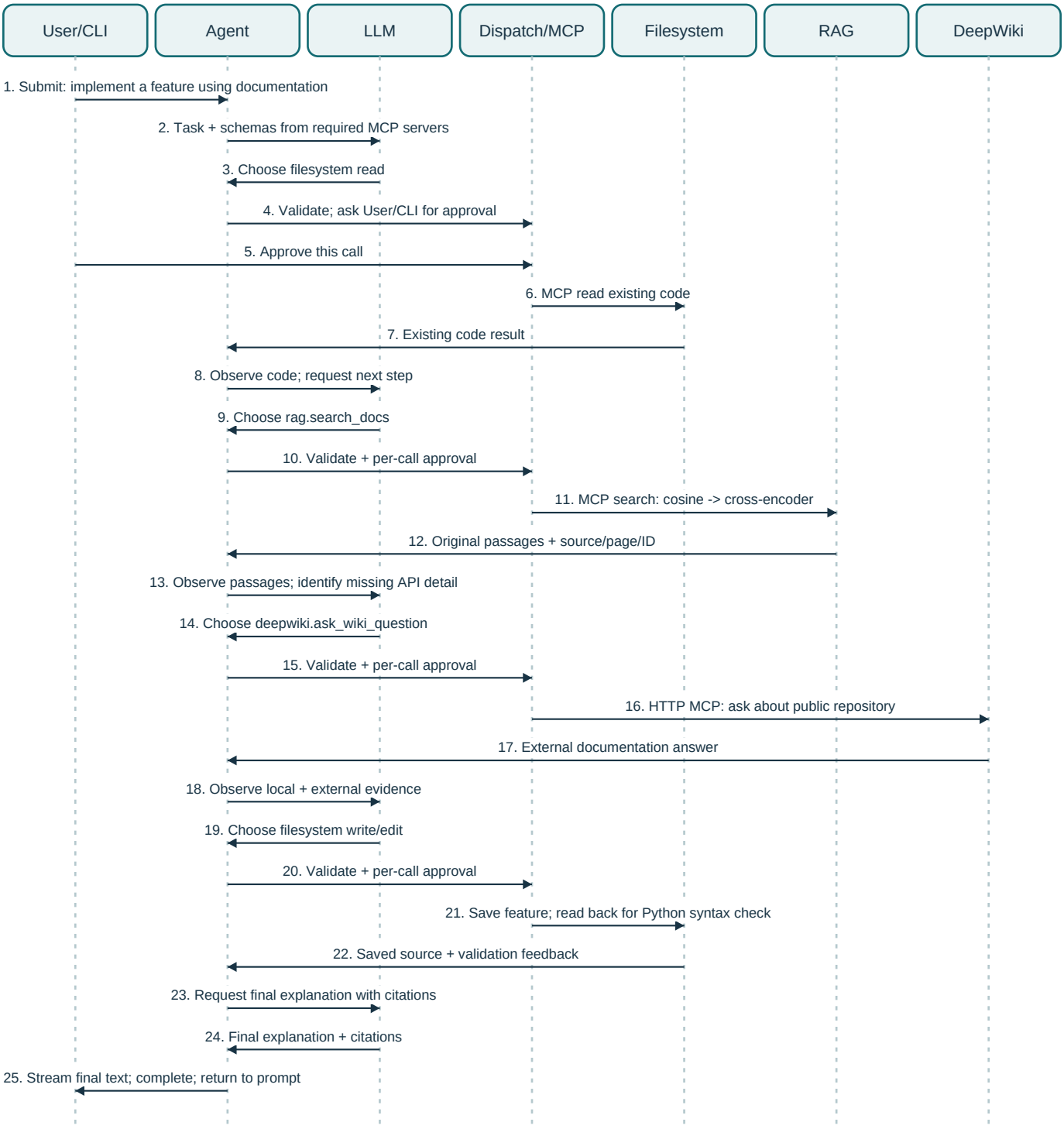


Syntax errors feed back for repair and block normal completion until resolved. The model can request the separate Python-check tool to run behavioral tests; those tests establish only what they check. Historical coding evaluation checks ran separately outside the model's writable fixture.

The optional Python-check call shown here was added after the recorded coding comparison. Test completeness and assertions determine what a passing check means.

Implement a documented feature

Confirmation mode | operation 3



Each valid call, including syntax-check reads, requests approval in confirmation mode. The model chooses tools and their order; this is an implementation walkthrough, not a claim that every live task succeeds. If local evidence is insufficient, retrieve another source or acknowledge the gap.